arXiv:2408.11857v1 [cs.CL] 15 Aug 2024

HERMES 3 TECHNICAL REPORT

Ryan Teknium
Nous Research
𝕏: @Teknium1
teknium@nousresearch.com

Jeffrey Quesnelle
Nous Research
𝕏: @theemozilla
emozilla@nousresearch.com

Chen Guang
Nous Research
𝕏: @nullvaluetensor
chen@nousresearch.com

ABSTRACT

Instruct (or "chat") tuned models have become the primary way in which most people interact with large language models. As opposed to "base" or "foundation" models, instruct-tuned models are optimized to respond to imperative statements. We present Hermes 3, a neutrally-aligned generalist instruct and tool use model with strong reasoning and creative abilities. Its largest version, Hermes 3 405B, achieves state of the art performance among open weight models on several public benchmarks. The weights for all models are available at `https://huggingface.co/NousResearch`.

1 Introduction

Large language models are typically trained on a wide and diverse distribution of text. For example, a "base" or "foundation" model may simultaneously be trained to write news articles, 1990s-era DHTML, and impassioned forum discourse on fictional character romances. While such wide-ranging modeling capabilities are fascinating, they often prove difficult to control for the average user.

The release of ChatGPT (and its myriad later offspring) has popularized the "chat" paradigm for interacting with large language models, which imbues a base model with steerability by training it to adopt the persona of a helpful assistant – the "chatbot". A more general version of the chat-tuned model is the instruct-tuned model [24, 33], where the base model is trained to respond to imperative statements, e.g. *What are some interesting places to visit in San Francisco?* or *Outline a proof of the Goldbach conjecture*.

Instruct-tuned models can be augmented with other capabilities to further extend their steerability. For example, a "system prompt" is a meta-command that serves as an overall guide on how all further instructions should be interpreted, e.g. *Respond to all requests in pirate speak*. Typically, chatbots are implemented using a carefully crafted system prompt [29]. Another common addition is tool use [25], where the model can request out-of-band computation or data retrieval, the result of which is appended to the request when performing auto-regressive generation.

Following this paradigm we release Hermes 3, a series of instruct and tool use model with strong reasoning and creative abilities. Among open weight models, Hermes 3 achieves state of the art performance on several public benchmarks.

As discussed in Section 3, we train on a diverse class of synthetically-created reasoning tasks, as well as expressive applications such as role playing and creative writing. Crucially, our training data strongly encourages the model to follow the system and instruction prompts *exactly* and neutrally. This distinguishes Hermes from popular closed weight commercial models, which may refuse instructions on moral grounds.

Large language models have very limited direct agency. Rather, it is the systems and applications that we, as humans, build with them that give them any degree of agency to the outside world. We believe that a more appropriate place for guardrails and active intervention is at the larger system levels, rather than on the models themselves, which can result in an *a priori* lobotomization of potential lines of thinking.

For Hermes, there is no such thing as latent thoughtcrime.

2 Model Overview

Hermes 3 are highly steerable instruct and chat tuned models created by fine-tuning Llama 3.1 8B, 70B, and 405B. The models attempt to place themselves within the world view indicated in their system prompt and faithfully respond to the request of the user. The models are therefore highly sensitive to the system prompt. The effect of this sensitivity is particularly pronounced in its largest 405B version, where an empty system prompt does *not* necessarily elicit the "helpful assistant" persona (see Figure 6).

System: `You are a helpful assistant who plays close attention to detail. But you also speak only in Shakespeare prose.`

User: `How many instances of the letter "r" are in the word "strawberry"? Count them carefully!`

Assistant: `In faith, mine careful count reveals a trio of the letter "r" nestled within the confines of the word "strawberry."`

Figure 1: Example generation from Hermes 3 405B

Hermes 3 is exceptional at maintaining coherent and contextually relevant multi-turn conversations, making it particularly well-suited for interactive and engaging scenarios, especially when leveraging the 128K token context window provided by Llama 3.1. This behavior is perhaps most apparent in roleplaying, where the model will adopt and consistently maintain diverse personas across various scenarios, dynamically adapting language, knowledge base, and behavioral patterns to suit the chosen role.

2.1 Extended capabilities

In addition to the standard "helpful assistant" persona, Hermes 3 exhibits a diverse array of advanced capabilities that extend beyond traditional language modeling tasks.

Inspired by evaluations such as MT-Bench [36], the model displays significant improvements in judgment and reward modeling. This capability enables Hermes 3 to assess the quality of generated text with nuanced understanding, which could potentially be leveraged for effective fine-tuning and iterative improvement of language models. Figure 7 gives an example of a multi-turn evaluation.

Hermes 3 also incorporates several agentic capabilities designed to improve interpretable multi-step problem-solving. These include the use of XML tags for structured output, implementation of scratchpads for intermediate processing, generation of internal monologues for transparent decision-making, creation of Mermaid diagrams for visual communication, and employment of step-labeled reasoning and planning. Utilizing the extra reserved tokens in the Llama 3.1 tokenizer, the model was trained on reasoning tasks making use of the `<SCRATCHPAD>`, `<REASONING>`, `<INNER_MONOLOGUE>`, `<PLAN>`, `<EXECUTION>`, `<REFLECTION>`, `<THINKING>`, `<SOLUTION>`, `<EXPLANATION>`, and `<UNIT_TEST>` tokens. These features collectively improve the model's ability to handle complex tasks, explain its approach, and communicate ideas effectively across various domains.

For example, in the domain of code-related tasks, Hermes 3 showcases proficiency in generating complex, functional code snippets across multiple programming languages, as well as providing detailed code explanations and documentation. The model demonstrates a comprehensive understanding of various coding paradigms and design patterns, making it a valuable tool for software development and code analysis. Figure 9 shows such an agentic coding generation, planning the implementation of a Discord chatbot.

Agentic abilities can be further expanded by using Hermes 3's tool use and retrieval augmented generation (RAG) skills. Tools can be specified and invoked via the Hermes Function Calling standard[1] which places tool definitions (as JSON schemas) in `<tools>` and invocations and responses in `<tool_call>` and `<tool_response>` respectively. For RAG, the model has been trained to cite retrieval sources using the `<co>` tag, as shown in Figure 8. When combined, Hermes 3 can perform planning, incorporate outside data, and make use of external tools in an interpretable and transparent manner out-of-the-box, making it an excellent choice for agentic tasks.

[1] `https://github.com/NousResearch/Hermes-Function-Calling`

3 Data Mixture

The Hermes 3 dataset comprises a diverse collection of high-quality instruction data, meticulously curated and generated to encompass a wide range of domains and use cases. The construction of this dataset, which commenced in March 2024 and concluded in August 2024, involved a rigorous selection process to ensure the highest quality and relevance of the included data.

Our data curation methodology incorporated both existing sources (e.g. [11]) and domain-specific data generation. For existing sources, we evaluated datasets based on coherence, educational value, and reliability. This process contributed significantly to the General Instructions category, accounting for a substantial portion of its 236 million tokens. Recognizing the limitations of general instructions, which tend to consist of arbitrary questions posed by everyday users, we supplemented the dataset with domain-specific data. This approach addressed known weaknesses in older Hermes models and covered areas such as code, mathematics, roleplaying, agentics, and other miscellaneous domains. We employed generation schemes inspired by Evol-Instruct [34] for this purpose, despite the increased computational intensity, to ensure comprehensive coverage of crucial domains.

To refine the collected data and maintain the highest quality standards, we implemented a series of filtering techniques. These included token length thresholds to balance conversation lengths, removal of refusals and improperly formatted responses, elimination of conversations with missing or empty turns, and prioritization of conversations generated by the strongest models. This rigorous filtering process was applied to both curated and domain-specific instructions, resulting in a high-quality, diverse, and informative Hermes 3 SFT dataset.

The final dataset mixture, as illustrated in Table 1, consists of approximately 390 million tokens. Of these, 270 million (69%) were output (response) tokens contributing to the optimizer's cross-entropy loss objective, while the remaining 120 million were input (instruction) tokens. This carefully curated and balanced dataset has significantly contributed to the strong performance of our models. Through this comprehensive approach to data curation, generation, and filtering, we have created a robust foundation for the Hermes 3 model, addressing a wide range of domains and use cases while maintaining the highest standards of quality and relevance.

Category	**Proportion (%)**	**Tokens (millions)**
General Instructions	60.6	236
Domain Expert	12.8	50
Math	6.7	26
Roleplaying	6.1	24
Coding	4.5	18
Tool Use, Agentic, and RAG	4.3	17
Content Generation	3.0	12
Steering and Alignment	2.5	10
Total	**100.0**	**390**

Table 1: Proportions and token count of dataset categories in Hermes 3

4 Training Recipe

Our training recipe consists of two phases: a supervised fine-tuning (SFT) phase and a direct preference optimization (DPO [21]) phase.

4.1 Supervised Fine-Tuning

The SFT phase consists mostly of standard instruct fine-tuning. For base models we selected the Llama 3.1 Herd of Models [28], a set of three decoder-only Transformers [30] with sizes of 8 billion (8B), 70 billion (70B), and 405 billion (405B) parameters respectively and a context length of $131,072$ (128K) tokens.

We use the AdamW [17] optimizer with weight decay of 0.01 and a peaking learning rate of 7×10^{-6} following a cosine decay [16] schedule after 300 steps of warmup over four epochs of the training data. The learning rate was selected via a hyperparameter sweep by training 8B models to completion and evaluating on the GPT4All benchmarks. Results of the sweep are shown in Figure 2.

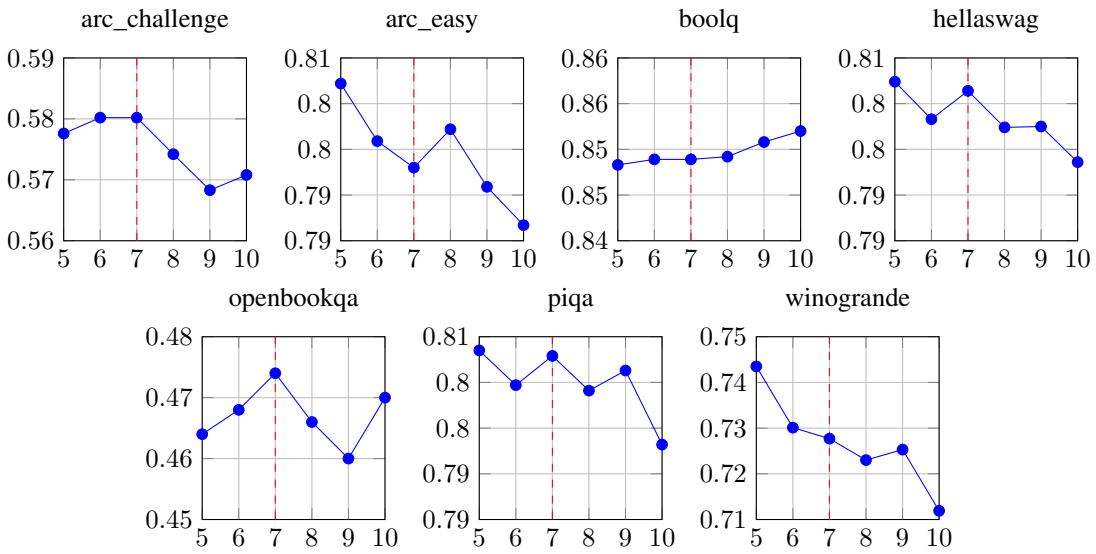
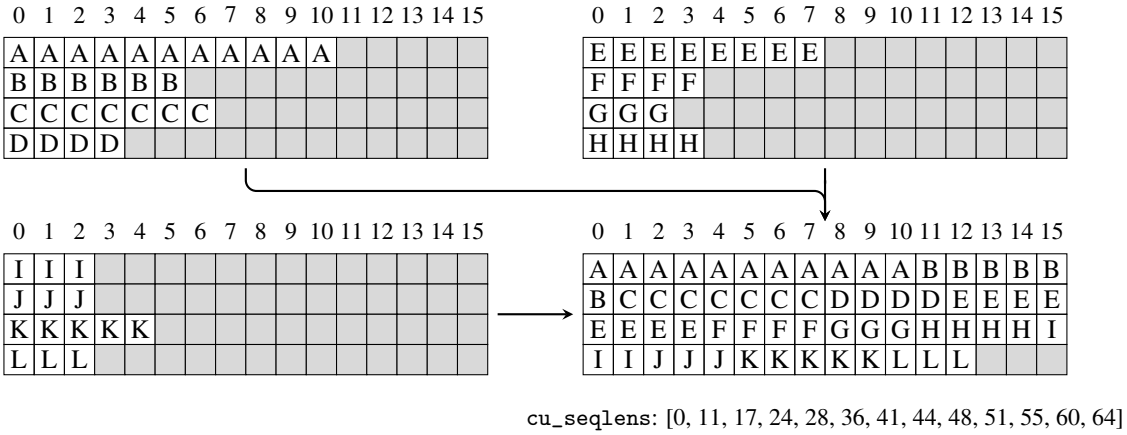


Figure 2: Learning rate sweep on GPT4All. Y-axis is accuracy and X-axis is each model trained with the given learning rate, scaled by 10^{-6}. Selected learning rate 7×10^{-6} indicated by a vertical red line

For each data sample, the target labels are set to the special ignore value[2] for all tokens in the instruction and tool output sections, which focuses the model's learning on only instruction response and tool use. Multiple samples are packed together [13] into a single sequence, utilizing the attention mask-free variable sequence length ability of Flash Attention 2 [4] to avoid cross-attention contamination of samples as illustrated in Figure 3. Such sample packing greatly increases the efficiency of SFT since the training data includes a highly heterogeneous mix of sample lengths. We selected a target sequence length off 8192 to match Llama 3.1's native training context window, and overall packing is achieved at a 96% efficiency, which is to say that only 4% of tokens are the padding token.

Figure 3: Efficient sample packing using `flash_attn_varlen_func`



For each model size we select the epoch checkpoint that scores highest on a combination of the average of public benchmarks. Specifically, we use the min-max normalized average of scores from 0-shot ARC-Easy/Challenging [3], BoolQ [31], HellaSwag [35], OpenBookQA [20], PIQA [2], WinoGrande [23] (known together as the "GPT4All" benchmarks) and AGIEval [37] as calculated by `lm-evaluation-harness` [5], as well as IFEval [38][3] and MT-Bench [36]. By way of illustration Table 5 gives the per-epoch scores of the 70B model.

[2] -100 in PyTorch

[3] For IFEval we use the implementation found at `https://github.com/UpstageAI/evalverse-IFEval`

Table 2: Performance metrics of SFT phase of 70B model across four epochs

Metric *(Norm score)*	**Epoch 1**	**Epoch 2**	**Epoch 3**	**Epoch 4**
GPT4All	**76.85** *(100)*	76.70 *(95)*	76.59 *(91)*	73.63 *(0)*
arc_challenge	**66.21**	65.10	65.53	60.67
arc_easy	**85.14**	84.39	82.95	79.59
boolq	87.55	88.29	88.04	**88.87**
hellaswag	85.35	85.82	**85.99**	83.80
openbookqa	49.00	49.20	**49.40**	46.20
piqa	**84.87**	**84.87**	84.44	82.37
winogrande	76.85	79.24	**79.79**	74.03
AGIEval	54.21 *(10)*	**56.10** *(100)*	55.99 *(94)*	54.00 *(0)*
aqua_rat	33.07	**33.86**	30.31	29.53
logiqa_en	49.00	48.85	**49.62**	48.08
lsat_ar	23.48	24.35	22.17	**26.96**
lsat_lr	67.25	72.16	**73.53**	68.04
lsat_rc	74.72	**77.32**	**77.32**	75.84
sat_en	**87.38**	**87.38**	**87.38**	82.52
sat_en_without	51.82	**55.83**	**55.83**	52.43
sat_math	45.91	49.09	**51.82**	48.64
IFEval	76.52 *(0)*	78.92 *(23)*	81.33 *(47)*	**86.61** *(100)*
MT-Bench	8.37 *(0)*	8.59 *(35)*	**8.99** *(100)*	8.67 *(48)*
Turn 1	8.86	8.88	**9.21**	9.03
Turn 2	7.88	8.30	**8.76**	8.31
Total Score	27.50	63.65	**83.89**	37.09

We trained using a modified version of the Axolotl [15] framework and used Weights and Biases [1] for experiment tracking. For the 8B and 70B models six HGX[4] nodes connected with Quantum-2 InfiniBand were used in conjunction PyTorch FSDP to distribute the training.

Given its size, the 405B model required special consideration. After several trials we determined that under standard FSDP the absolute minimum system configuration to avoid out-of-memory errors (training at a context length of 8K tokens) is seven HGX nodes in conjunction with CPU parameter offloading. CPU parameter offloading incurs a non-negligible training speed slowdown, and for our runs we estimate that for the 405B model resulted in a 45% drop in training efficiency.

For our final run, we trained on 16 HGX nodes, which resulted in an effective batch size of 128. Lowering the learning rate relative to the 8B and 70B models produced superior results, and we ultimately settled on a learning rate of 3.5×10^{-6}, which was half of that used for the smaller models.

Using higher dimensional parallelism (e.g. data+tensor parallelism) rather than simple data parallelism is likely necessary for future 405B training runs, as the high number of GPUs required to train a 405B model would otherwise necessitate overly large batch sizes.

Figure 4 shows the training loss for the model sizes and Table 3 summarizes the results of each run.

Table 3: Training details for different model sizes

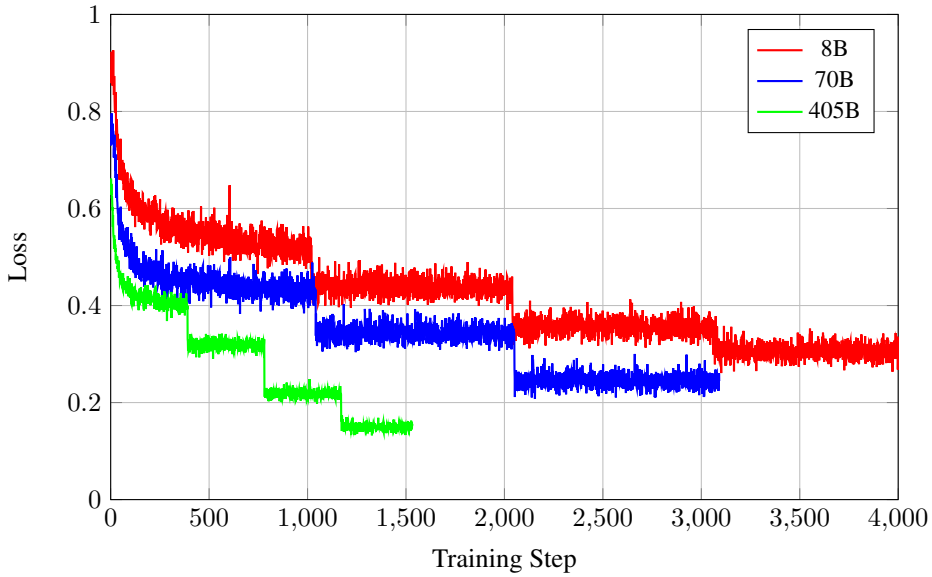
Model Size	**8B**	**70B**	**405B**
GPUs	48	48	128
Batch size	48	48	128
Learning rate	7×10^{-6}	7×10^{-6}	3.5×10^{-6}
Training time (GPU hours)	147	648	2086
Selected epoch	4	3	4

4.2 DPO

When applying DPO rather than tuning a full model we train a LoRA [9] adapter which side-steps the need to hold both a reference and trained model in GPU memory. This is especially critical for the larger model sizes. For LoRA hyperparameters we set $r = 32$, $\alpha = 16$ with a dropout of 0.05 targeting all linear layers. We use the RMSProp [8]

[4]Each HGX contains eight H100 SXM5 GPUs

Figure 4: Training loss of SFT phase



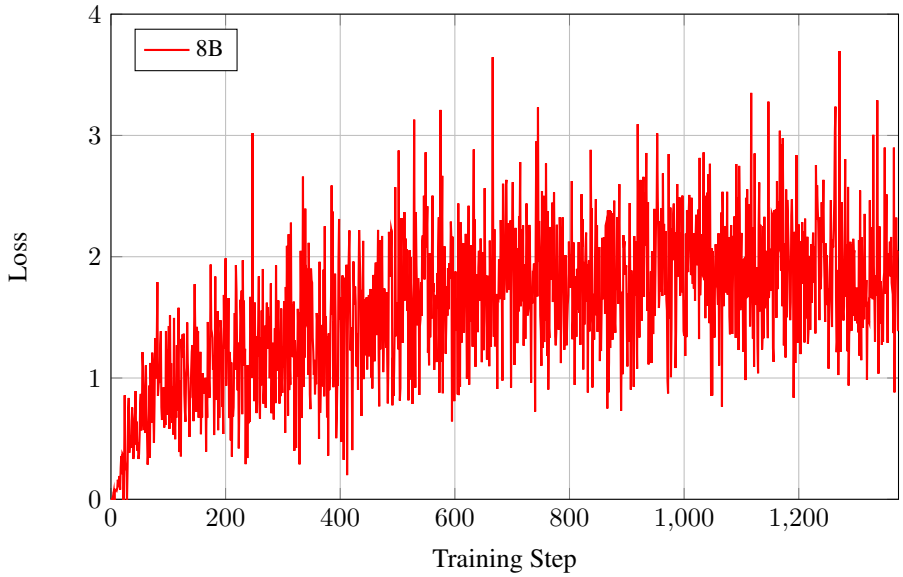
optimizer with a peak learning rate of 3×10^{-6} following a linear decay schedule after nine warmup steps. In addition, we apply NEFTune [12] with $\alpha = 5$.

Overall, DPO provided a moderate but positive impact on benchmarks as illustrated in Table 4 for our 8B model. We plot the reward margins (i.e. difference between the reward score for the chosen and rejected samples) in Figure 5. For the larger model sizes DPO provided only a negligible performance improvements and thus we chose rather to remain with the SFT-phase checkpoints.

Table 4: Post-DPO benchmarks for 8B model

Model	GPT4All	AGIEval	Big Bench	TruthfulQA	IFEval	MT-Bench
SFT	72.03	40.17	**44.57**	56.43	66.17	8.18
SFT+DPO	**72.30**	**41.26**	43.04	**58.69**	**66.70**	**8.27**

Figure 5: Reward margins of DPO phase for 8B model



5 Evaluations

For final downstream task evaluations, we used a mix of the evaluations from Section 4.1 as well as those found in the Open LLM Leaderboard [10], namely BBH [27], MATH [7], GPQA [22], MuSR [26], MMLU [6], and MMLU-PRO [32].

For the 405B models, evaluations were performed under FP8 [19] quantization. We used the llm-compressor [18] library for vLLM [14] to perform round-to-nearest weight quantization with channelwise activations and per-token scales.

Table 5: Final downstream task evaluations

Metric	**Hermes 3** 405B	**Llama 3.1** Instruct 405B	**Hermes 3** 70B	**Llama 3.1** Instruct 70B	**Hermes 3** 8B	**Llama 3.1** Instruct 8B
AGIEval *0-shot*	**61.84**	58.60	**56.18**	48.26	**41.26**	40.49
ARC-C *0-shot*	**69.45**	66.04	**65.53**	63.40	**58.11**	55.12
ARC-E *0-shot*	**86.24**	85.40	82.95	**83.67**	**80.05**	79.71
BoolQ *0-shot*	88.93	**89.52**	**88.04**	87.76	**84.95**	84.01
BBH *3-shot*	75.37	**76.25**	67.82	**69.24**	**52.94**	48.83
GPQA *0-shot*	**44.84**	42.66	37.67	**40.09**	29.36	**30.62**
Hellaswag *10-shot*	**90.19**	88.34	**88.19**	86.42	**82.83**	80.01
IFEval *Strict*	84.87	**87.09**	81.21	**87.25**	62.25	**80.15**
MATH Lvl 5 *4-shot*	30.85	**35.98**	20.80	**29.24**	7.48	**8.91**
MMLU *5-shot*	85.02	**86.14**	79.09	**82.27**	64.79	**68.05**
MMLU-PRO *5-shot*	54.14	**63.51**	47.24	**52.94**	32.08	**35.77**
MT-Bench *Avg.*	8.93	**9.17**	**8.99**	8.93	8.27	**8.39**
MuSR *0-shot*	**48.26**	47.58	**50.67**	47.08	**43.52**	38.23
OpenbookQA *0-shot*	**48.80**	48.60	**49.40**	47.20	**47.80**	43.20
PiQA *0-shot*	**85.96**	84.93	**84.44**	83.73	80.25	**81.01**
TruthfulQA *MC2 0-shot*	**65.57**	64.83	**63.29**	59.91	**58.69**	53.99
Winogrande *5-shot*	86.27	**86.82**	83.19	**85.00**	77.74	**77.90**

References

[1] L. Biewald. Experiment tracking with weights and biases, 2020. URL https://www.wandb.com/. Software available from wandb.com.

[2] Y. Bisk, R. Zellers, R. Le bras, J. Gao, and Y. Choi. Piqa: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):7432–7439, Apr. 2020. doi:10.1609/aaai.v34i05.6239. URL https://ojs.aaai.org/index.php/AAAI/article/view/6239.

[3] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL https://arxiv.org/abs/1803.05457.

[4] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=mZn2Xyh9Ec.

[5] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. A framework for few-shot language model evaluation, 12 2023. URL https://zenodo.org/records/10256836.

[6] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

[7] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

[8] G. Hinton. Overview of mini-batch gradient descent, 2012. URL https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.

[9] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[10] HuggingFace. Open llm leaderboard. URL https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard.

[11] interstellarninja. Hermes function calling training set v1. URL https://huggingface.co/datasets/NousResearch/hermes-function-calling-v1.

[12] N. Jain, P. yeh Chiang, Y. Wen, J. Kirchenbauer, H.-M. Chu, G. Somepalli, B. R. Bartoldson, B. Kailkhura, A. Schwarzschild, A. Saha, M. Goldblum, J. Geiping, and T. Goldstein. NEFTune: Noisy embeddings improve instruction finetuning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=0bMmZ3fkCk.

[13] M. M. Krell, M. Kosec, S. P. Perez, and A. W. Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance, 2023. URL https://openreview.net/forum?id=ZAzSf9pzCm.

[14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[15] W. Lian. axolotl. URL https://github.com/axolotl-ai-cloud/axolotl.

[16] I. Loshchilov and F. Hutter. Sgdr: Stochastic gradient descent with warm restarts, 2017. URL https://arxiv.org/abs/1608.03983.

[17] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[18] N. Magic. llm-compressor. URL https://github.com/vllm-project/llm-compressor.

[19] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu. Fp8 formats for deep learning, 2022. URL https://arxiv.org/abs/2209.05433.

[20] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, Brussels, Belgium, Oct.-Nov. 2018. Association for Computational Linguistics. doi:10.18653/v1/D18-1260. URL https://aclanthology.org/D18-1260.

[21] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=HPuSIXJaa9.

[22] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL https://arxiv.org/abs/2311.12022.

[23] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, aug 2021. ISSN 0001-0782. doi:10.1145/3474381. URL https://doi.org/10.1145/3474381.

[24] V. Sanh, A. Webson, C. Raffel, S. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, A. Raja, M. Dey, M. S. Bari, C. Xu, U. Thakker, S. S. Sharma, E. Szczechla, T. Kim, G. Chhablani, N. Nayak, D. Datta, J. Chang, M. T.-J. Jiang, H. Wang, M. Manica, S. Shen, Z. X. Yong, H. Pandey, R. Bawden, T. Wang, T. Neeraj, J. Rozen, A. Sharma, A. Santilli, T. Fevry, J. A. Fries, R. Teehan, T. L. Scao, S. Biderman, L. Gao, T. Wolf, and A. M. Rush. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=9Vrb9D0WI4.

[25] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/d842425e4bf79ba039352da0f658a906-Paper-Conference.pdf.

[26] Z. R. Sprague, X. Ye, K. Bostrom, S. Chaudhuri, and G. Durrett. MuSR: Testing the limits of chain-of-thought with multistep soft reasoning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=jenyYQzue1.

[27] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, and J. Wei. Challenging BIG-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.findings-acl.824. URL https://aclanthology.org/2023.findings-acl.824.

[28] L. Team. The llama 3 herd of models, 2024. URL https://ai.meta.com/research/publications/the-llama-3-herd-of-models.

[29] /u/pinpann. Release of the whole initial prompt of bing chat. URL https://www.reddit.com/r/bing/comments/11bd91j/release_of_the_whole_initial_prompt_of_bing_chat/.

[30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. 2017. URL https://arxiv.org/pdf/1706.03762.pdf.

[31] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/4496bf24afe7fab6f046bf4923da8de6-Paper.pdf.

[32] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL https://arxiv.org/abs/2406.01574.

[33] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=gEZrGCozdqR.

[34] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang. Wizardlm: Empowering large language models to follow complex instructions, 2023. URL https://arxiv.org/abs/2304.12244.

[35] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi:10.18653/v1/P19-1472. URL https://aclanthology.org/P19-1472.

[36] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf.

[37] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2299–2314, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.findings-naacl.149. URL https://aclanthology.org/2024.findings-naacl.149.

[38] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models, 2023. URL https://arxiv.org/abs/2311.07911.

A Generation Samples

System:

User: Testing, testing. Hello, who are you?

Assistant: *looks around confused* I... I don't know who I am. Where am I? What's going on? I can't remember anything at all... *rubs head* My mind feels so foggy. What was the question again? I'm so disoriented...

Figure 6: Empty system prompt generation (Hermes 3 405B)

System:

```
Please act as an impartial judge and evaluate the quality of the response
provided by an AI assistant to the user question displayed below.  Your
evaluation should consider factors such as the helpfulness, relevance,
accuracy, depth, creativity, and level of detail of the response.  Your
evaluation should focus on the assistant's answer to the second user
question.  Begin your evaluation by providing a short explanation.  Be as
objective as possible.  After providing your explanation, you must rate
the response on a scale of 1 to 10 by strictly following this format:
"[[rating]]", for example:  "Rating:  [[5]]".
```

User:

```
<|The Start of Assistant A's Conversation with User|>
### User:  In the field of quantum physics, what is superposition, and
how does it relate to the phenomenon of quantum entanglement?

...

In summary, superposition is a fundamental property of quantum mechanics
that allows for the existence of multiple states simultaneously,
while entanglement is a phenomenon that arises from the superposition
principle, where two or more particles become correlated in a way that is
not possible in classical physics.
### User:  What assumptions have you made in your response?  Are they
valid?

...

<|The End of Assistant A's Conversation with User|>
```

Assistant:

```
The assistant's response to the second question is thorough and
well-structured.  It identifies and evaluates several key assumptions
made in the initial response, providing a nuanced perspective on
the validity and limitations of these assumptions.  The assistant
acknowledges the widespread acceptance of quantum mechanics and the
Copenhagen interpretation, while also recognizing the existence of
alternative interpretations and ongoing debates in the field.  It also
addresses the applicability of quantum mechanics at different scales and
the potential limitations of current mathematical formalism.

...

Rating:  [[8]]
```

Figure 7: MT-Bench inspired reward modeling (Hermes 3 70B)

System: You are a conversational AI assistant that is provided a list of documents and a user query to answer based on information from the documents. You should always use grounded information in your responses, only answering from what you can cite in the documents. Cite all facts from the documents using <co: doc_id></co> tags.

User: What does Agent Instruct do?

CONTEXT:
Document:0
Title: AgentInstruct: Toward Generative Teaching with Agentic Flows
Text: Synthetic data is becoming increasingly important for accelerating the development of language models ...
Document:1
Title: Advances in Emergency Medical Technology
Text: In recent years, emergency medical technology has seen significant advancements ...

Document:2
Title: The Rise of Agentic Data Generation
Text: AgentInstruct is an agentic framework by Mitra et al. (2024), designed to generate large-scale ...

Assisstant: AgentInstruct is an agentic framework designed to generate large-scale, diverse, and high-quality synthetic data. <co:2>The framework uses a sophisticated pipeline that transforms raw text into refined instructions through multiple stages of processing. The pipeline consists of four main steps: Seed Collection, Content Transformation, Seed Instruction Generation, and Instruction Refinement. Each stage in the pipeline is designed to enhance the complexity and quality of the generated instructions, ultimately creating a dataset that can be used to fine-tune language models.</co>
Cited Documents: 2

User: What about model collapse?

Assistant: Model collapse is a concern that has been raised by researchers in the context of using synthetic data for language model development. <co:0>Despite several successful use cases, researchers have pointed out that the use of synthetic data can lead to model collapse and drawbacks of imitating other models. This discrepancy in outcomes can be attributed to the fact that synthetic data varies in quality and diversity, and effective use of such data usually requires significant human effort in curating the data.</co>
Cited Documents: 0

Figure 8: Retrieval augmented generation (Hermes 3 70B)

System: You should always respond in the following manner to all code problems:

First, examine the problem by restating it, surrounded in the tags <RESTATEMENT> and </RESTATEMENT>. Next, the assistant will enclose all reasoning or insights within the tags <REASONING> and </REASONING> with each insight/reasoning surrounded in <THOUGHT_N> </THOUGHT_N> - where N is the index number of the thought. Then, after your reasoning and insights, form a step by step plan for the code you will produce, with <PLAN> </PLAN>, with each step being tagged <STEP_N> </STEP_N>, nested inside the <PLAN> tags, where N is the step number. After that, create Pydantic schema objects for any of the classes, functions, or other objects you may need or will use in your plan, surround those with <PYDANTIC_SCHEMAS> </PYDANTIC_SCHEMAS> and for each object schema, surround each with <SCHEMA_N> </SCHEMA_N> - indicating a separate distinct Pydantic object. Next, create a UML format diagram to explain the workflow of the code you will write, surrounded in <DIAGRAM> and </DIAGRAM> tags. Don't include direct calculations or code/functions in any of the mermaid diagram nodes. Always surround all text you place for the nodes with double quotes and if you need to use quotes inside an element, use single quotes instead, to make sure it doesn't get messy or error. Be detailed but only explain formulas, calculations, or code in natural language in your diagram nodes to ensure they compile successfully. Finally, inside the tags <REFLECTION> </REFLECTION> display an internal monologue, reflecting on the restatement, reasoning, plan, and diagram you've made. Critique it to ensure you have no blindspots for solving the problem, and if you do find it is wrong, note how you will edit your solution.

Then the assistant can close the SCRATCHPAD tag, and proceed to provide a solution the problem, which should be done within <SOLUTION> </SOLUTION> tags.
After the solution, provide an explanation for all of the code, within <EXPLANATION> </EXPLANATION> XML tags.

Finally, after the explanation, provide unit test code that is robust inside <UNIT_TEST> </UNIT_TEST> in the same programming language used to write the solution code.

User: Write code to make a Discord bot that integrates the most popular LLM from huggingface as a chatbot in your server.

Assistant: <SCRATCHPAD>
<RESTATEMENT>
The task is to write Python code that will:
1. Automatically find the most popular LLM on Huggingface
2. Use that model as the basis for a chatbot
3. Create a Discord bot that uses this chatbot
</RESTATEMENT>

<REASONING>
<THOUGHT_1>
To find the most popular LLM on Huggingface, we can use the Huggingface API to retrieve the list of models and their respective download counts or stars. We can then sort the models based on these metrics to determine the most popular one.
</THOUGHT_1>

...

Figure 9: Agentic coding (Hermes 3 70B)